



Prompting Techniques

Prompting Techniques

1) Zero-Shot Prompting

What it is

Give only the task instructions (no examples). The model relies on its prior training.

Use when

- Tasks are simple/standard (classification, Q&A, short summaries)
- You need speed and low token cost

How to write

- State the goal in one sentence
- Specify audience, tone, and format
- Add hard constraints (length, sections)

Pros

- Fast, cheap, easy to iterate

Cons

- Inconsistent on complex/novel tasks

Common pitfalls

- Vague verbs ("talk about...")
- Missing format/length constraints

Examples

Task: Summarize the article below for a busy executive in 1 20–150 words.

Tone: neutral, non-promotional.

Format: 3 bullets, then 1-paragraph summary.

Text: <<<...>>>

Classify each review as {Positive|Neutral|Negative}. Return JSON array of {id, label}.

Reviews: <<<...>>>

2) Step-Back Prompting

What it is

Ask a general, higher-level question first to activate relevant principles, then solve the specific task.

Use when

- You need grounded answers (principles > details)
- Creative planning, strategy, pedagogy, UX/architecture decisions

How to write

1. *Step back*: extract general criteria or a framework
2. Apply it to the concrete request

Pros

- Reduces bias; improves structure and coverage

Cons

- Two steps = more tokens/time

Pitfalls

- Skipping the application step (i.e., never uses the extracted principles)

Examples

Step back: List key criteria for engaging first-person shooter levels (bullets).

Apply: Using those criteria, draft a 3-mission storyline set in a retro neon city.

Step back: What are best practices for onboarding forms (accessibility + conversion)?

Apply: Redesign the sign-up flow for a grocery delivery app in 5 steps.

3) One-Shot & Few-Shot Prompting

What it is

Provide 1 (one-shot) or several (few-shot: 3–5) labeled examples that demonstrate the exact input → output pattern.

Use when

- Precise format/style is critical
- Zero-shot results are inconsistent
- You're aligning to domain tone or schema

How to write

- Keep examples short, canonical, and diverse
- Put the **rules** before the examples
- End with a **clear new input**

Pros

- Big gains in formatting and tone adherence

Cons

- More tokens; risk of overfitting to examples

Pitfalls

- Inconsistent examples; mixing styles; leaking irrelevant details

Examples

Task: Convert product notes to a JSON spec.

Rules: Return only valid JSON {name, features[], priceUSD:n

```
umber}.  
Examples:  
Input: "Acme Pen..."  
Output: {"name":"Acme Pen","features":["refillable","fine tip"],"priceUSD":12.99}  
---  
Input: "Nova Mug..."  
Output: {"name":"Nova Mug","features":["ceramic","350ml"],"priceUSD":9.5}  
Now convert:  
Input: "Orbit Bottle..."  
Output:
```

4) Chain-of-Thought (CoT) Prompting

What it is

Ask the model to reason step-by-step before giving the final answer.

Use when

- Math, logic, multi-step reasoning, non-obvious causality

How to write

- Request intermediate reasoning and a final boxed answer
- Optionally hide reasoning in UI while keeping it for QA

Pros

- Higher accuracy on complex tasks

Cons

- More tokens; may over-explain if not bounded

Pitfalls

- Letting reasoning ramble; not capping length

Examples

```
Solve the problem step by step. Show numbered reasoning, then "Final Answer: <value>".
```

Problem: ...

Think step by step. After reasoning, output exactly:
`{"final_answer": "<short answer>"}`

5) System Prompting

What it is

A top-level instruction that defines role, goals, constraints, safety, and output format for the whole session/call.

Use when

- You need consistent behavior across interactions
- Production agents/APIs

How to write

- State role, audience, do/don't rules, format contract, safety rails

Pros

- Strongest, most persistent control

Cons

- Long/ambiguous system prompts can cause conflicts

Pitfalls

- Redundant or contradictory rules vs. user prompt

Example (system)

You are a senior technical writer.
Always write concise, factual content at CEFR B2 reading level.
When asked for structured data, return only valid JSON; no prose.
If a request involves PII, refuse and suggest safe alternatives.

6) Role Prompting

What it is

Assign a persona (teacher, SWE, doctor, guide) to shape voice, assumptions, and domain focus.

Use when

- Tone/lexicon matters (industry-specific)
- You want domain heuristics without long instructions

How to write

- Name the role, level, and goals; specify audience and constraints

Pros

- Natural tone alignment; reduced over-explaining

Cons

- Risk of stereotype traps if under-specified

Pitfalls

- Vague roles ("expert") with no scope or audience

Examples

```
Act as a UX research lead. Audience: product managers.  
Goal: turn interview notes into 5 insights with evidence quotes.
```

```
You are a TypeScript mentor. Explain with small, runnable examples and add 1 pitfall per topic.
```

7) Contextual Prompting

What it is

Provide dynamic, task-specific background (docs, data, constraints) to ground the answer.

Use when

- RAG flows, data extraction, policy-bound responses, brand voice

How to write

- Delimit context clearly; instruct model to use only provided facts
- Tell it what to do if info is missing (e.g., "respond with null")

Pros

- Higher factuality; fewer hallucinations

Cons

- Token heavy; context must be curated

Pitfalls

- Unscoped dumps; not stating "do not infer beyond context"

Examples

Context (authoritative; do not use outside knowledge):

<<<DOCS>>>

...

</DOCS>

Task: Draft a 120-word abstract using only the context. If unknown, state "insufficient data."

8) Self-Consistency Prompting

What it is

Sample multiple reasoning paths at higher temperature, then majority-vote the final answer.

Use when

- Hard reasoning tasks where single CoT may fail

How to write

- Fix the prompt; run N samples (e.g., 5–20); select the most common final answer
- Optionally re-check the winner with a verification prompt

Pros

- Higher accuracy; robustness to spurious chains

Cons

- More cost/latency (multiple calls)

Pitfalls

- Not fixing `seed/temperature` ranges; not normalizing answer format

Example (controller logic)

```
Prompt (CoT). Params: temperature=0.8, top_p=0.95.  
Run 10 samples → extract "Final Answer:" → majority vote.  
If tie, run verifier: "Check arithmetic and constraints; re  
turn PASS/FAIL + corrected answer."
```

9) Tree of Thoughts (ToT) Prompting

What it is

Generalizes CoT to **branching** reasoning. Explore multiple partial solutions ("thoughts"), evaluate, and expand promising branches.

Use when

- Planning, puzzle solving, program synthesis, design exploration

How to write

- Specify breadth/depth limits and evaluation criteria
- Alternate *expand* → *evaluate* → *select* → *continue*

Pros

- Better exploration of solution space

Cons

- Controller complexity; higher cost

Pitfalls

- No scoring rubric; branches wander or explode

Example (controller prompts)

Expand: Generate 3 candidate next steps (Thought 1–3) toward the goal.

Evaluate: Score each 0–5 for plausibility, constraints, and progress.

Select: Keep top 2; continue for up to 3 depths. Then synthesize final plan.

10) ReAct Prompting

What it is

Reason-and-Act loop: the model alternates between thoughts, tool calls (actions), and observations from tools, until a final answer.

Use when

- Tool use: web search, code execution, DB queries
- Multi-step tasks requiring current or external data

How to write

- Define the **format**: `Thought:`, `Action:`, `Action Input:`, `Observation:`
- Provide allowed tools and when to use them
- Add a **FINAL_ANSWER** boundary

Pros

- Transparency; grounded in real observations

Cons

- Controller/tooling needed; can be verbose

Pitfalls

- Missing stop conditions; leaking tool outputs into user-facing text

Example

You may use these tools: {search, calculator}.

When you need information, write:

Thought: why the tool is needed

Action: search

Action Input: "query"
Observation: <tool result>

Iterate until ready. Then output:
FINAL_ANSWER: <concise answer>
Stop after FINAL_ANSWER.

11) Quick Selection Guide

Technique	Best For	Cost/Complexity	Key Tip
Zero-Shot	Simple, standard tasks	Low	Add explicit format + length
Step-Back	Principle-driven tasks	Low-Med	Always apply principles to the case
One/Few-Shot	Strict formats, domain tone	Med	Keep examples short & consistent
CoT	Math/logic reasoning	Med	Cap reasoning length; finalize clearly
System	Consistent behavior	Low	Define role, rules, and safety early
Role	Tone/domain alignment	Low	Name audience & success criteria
Contextual	Factual grounding	Med	"Use only context; return null if missing"
Self-Consistency	Hard reasoning	High	Majority vote with verifier
ToT	Exploration/planning	High	Breadth/depth + scoring rubric
ReAct	Tool-grounded tasks	High	Enforce Thought/Action/Obs/Final

12) Reusable Snippets (copy-paste)

Length & format contract

Write ≤ 180 words. Sections (exactly):
Summary

```
# Action Items (3 bullets)
```

Context-only guardrail

```
Use only facts inside <<CONTEXT>>...</CONTEXT>. If a fact is
missing, answer "insufficient data."
```

Few-shot skeleton

```
Rules: Return only valid JSON {question, answer}.
Example:
Input: "...
Output: {"question":"...", "answer":"..."}
Now process:
Input: "...
Output:
```

CoT finalization

```
Think step by step. Then output exactly:
Final Answer: <short answer>
```

ReAct stop

```
When done, output:
FINAL_ANSWER: <answer>
Stop after this line.
```

Key Takeaway

Choose the **simplest technique** that reliably meets your task's requirements. Combine **format contracts**, **context**, and—when needed—**reasoning/branching patterns** to turn LLMs from helpful assistants into **predictable, production-ready components**.